Quiz: Data Pipeline and Orchestration

Dương Nguyễn Thuận **Đinh Quang Vinh**

Câu hỏi 1 : Trong bức tranh tổng thể về rủi ro xử lý dữ liệu (Data Integration Iceberg), hiện tượng ứng dụng nguồn tự động thêm, xóa hoặc sửa đổi tên trường dữ liệu được gọi là gì và trụ cột quan sát nào trực tiếp nhận diện nó?

a) Data Duplication; được nhận diện bằng trụ cột Độ tươi (Freshness) của các bảng liên quan.

b) Silent Failure; được nhận diện bằng cách kiểm tra trụ cột Dung lượng (Volume) dữ liệu ghi nhận.

c) Schema Drift; được nhận diện thông qua trụ cột Schema và Lineage để theo dõi sự thay đổi cấu trúc và phạm vi ảnh hưởng.

d) Late-Arriving Data; được nhận diện thông qua cơ chế chính sách thử lại (Retry Policy).

Câu hỏi 2 : Theo các trụ cột về Data Observability trình bày trong tài liệu, trụ cột nào chịu trách nhiệm phát hiện các đường ống dữ liệu bị chậm trễ hoặc tắc nghẽn trước khi làm hỏng báo cáo?

a) Schema (Cấu trúc).

b) Freshness (Độ tươi).

c) Volume (Độ lớn).

d) Distribution (Sự phân bố).

Câu hỏi 3 : Vai trò quan trọng nhất của cột trụ 'Lineage' trong việc quản trị và quan sát hệ thống dữ liệu là gì?

a) Đo lường chính xác dung lượng lưu trữ còn trống trên các đĩa cứng của Data Warehouse.

b) Tự động ngăn chặn các truy vấn SQL kém tối ưu làm tiêu tốn tài nguyên hệ thống.

c) Đảm bảo mã hóa dữ liệu nhạy cảm theo các tiêu chuẩn an toàn bảo mật.

d) Cho biết luồng di chuyển của dữ liệu giữa các hệ thống, giúp xác định nguồn gốc sự cố và mức độ ảnh hưởng lan truyền.

Câu hỏi 4 : Trong một pipeline nạp dữ liệu (ingestion) phân vùng theo ngày, cơ chế xử lý nào giúp đảm bảo tính toàn vẹn lặp lại (Idempotence) khi thực thi lại pipeline nhiều lần trên cùng một phân vùng?

a) Chèn nối tiếp dữ liệu mới vào cuối bảng đích mà không cần kiểm tra dữ liệu hiện có (APPEND only).

b) Chạy lại toàn bộ dữ liệu từ quá khứ đến hiện tại (Full Reload) cho tất cả các bảng trong mọi chu kỳ lập lịch.

c) Thực hiện tự động cập nhật cấu trúc bảng bất cứ khi nào xuất hiện thêm trường dữ liệu mới từ nguồn.

d) Thực hiện chiến lược xóa dữ liệu cũ thuộc phân vùng tương ứng trước khi chèn dữ liệu mới (DELETE + INSERT per partition).

Câu hỏi 5 : Khi khởi tạo một pipeline xử lý theo ngày cho tập dữ liệu lịch sử, tại sao việc xác định mốc 'baseline cutover' (ví dụ chọn mốc bắt đầu quản lý từ 2013-01-01 dù dữ liệu gốc có từ 2012-07-04) lại là một quyết định kiến trúc quan trọng?

a) Tự động kích hoạt cơ chế nạp lại dữ liệu toàn phần (full reload) cho tất cả các bảng chiều (dimension tables).

b) Ép buộc bộ điều phối phải chạy tất cả các tác vụ dưới dạng xử lý song song không theo thứ tự phụ thuộc.

c) Tránh việc phải xử lý các trường hợp ngoại lệ cho những bảng dữ liệu nguồn xuất hiện muộn hơn trong lịch sử.

d) Giảm bớt thời gian cài đặt và đồng bộ hóa biến môi trường trên máy cục bộ của lập trình viên.

Câu hỏi 6 : Sự khác biệt cốt lõi trong cách tiếp cận kiến trúc giữa mô hình dựa trên tác vụ (Task-based như Apache Airflow) và dựa trên tài sản dữ liệu (Software-Defined Assets như Dagster) là gì?

a) Task-based chỉ hỗ trợ chạy thủ công, còn Asset-based bắt buộc phải chạy theo các Scheduler tự động.

b) Task-based tập trung định nghĩa công việc và thứ tự thực thi, trong khi Asset-based khai báo các sản phẩm dữ liệu cần có và tự suy ra quan hệ phụ thuộc.

c) Task-based chỉ hoạt động với các script Python nội bộ, trong khi Asset-based điều phối được các công cụ bên ngoài như dbt hay Snowflake.

d) Task-based quy định các tập dữ liệu kết quả cần có, còn Asset-based tự động tạo ra lịch trình thực thi.

Câu hỏi 7 : Trong Dagster, cơ chế nào cho phép một hàm được gắn decorator '@asset' tự động nhận biết quan hệ phụ thuộc (upstream dependencies) từ các asset khác?

a) Khai báo tên các asset phụ thuộc vào tham số 'deps' trong decorator '@asset' dạng chuỗi ký tự.

b) Cấu hình thứ tự thực thi thủ công bên trong tệp 'dagster.yaml' của dự án.

c) Thêm một decorator phụ '@dg.dependency' phía trên định nghĩa hàm asset.

d) Đặt tên các tham số đầu vào của hàm trùng khớp với tên của các asset tiền nhiệm.

Câu hỏi 8 : Chính sách thử lại ('RetryPolicy') được áp dụng cho 9 asset nạp dữ liệu phân vùng theo ngày trong dự án có cấu hình chi tiết như thế nào?

a) Thử lại không giới hạn số lần với khoảng hoãn cố định 10 giây cho đến khi thành công.

b) Thử lại 2 lần với khoảng thời gian giảm dần theo cấp số cộng.

c) Chỉ thử lại khi sự cố phát sinh từ các câu lệnh SQL biến đổi trong dbt.

d) Thử lại tối đa 3 lần, thời gian hoãn ban đầu 30 giây và tăng dần theo chiến lược lùi mũ ('EXPONENTIAL').

Câu hỏi 9 : Mô hình dbt incremental 'mart_daily_sales_features' xử lý bài toán dữ liệu trả hàng đến trễ (late-arriving returns) bằng chiến lược tính toán nào?

a) Xóa toàn bộ dữ liệu trùng lặp trong bảng staging trước khi ghi đè vào bảng mart.

b) Chỉ cập nhật duy nhất các bản ghi của đúng ngày phân vùng hiện tại.

c) Tính toán và xử lý lại một cửa sổ cuộn 45 ngày (rolling 45-day window) trong mỗi lần chạy.

d) Thực hiện tính toán lại toàn bộ lịch sử dữ liệu (Full Refresh) ở mỗi lần phân vùng chạy.

Câu hỏi 10 : Cảm biến cảnh báo lỗi chạy pipeline ('failure_email_alert') được tích hợp trong dự án kích hoạt hành động gửi email khi nào?

a) Kích hoạt khi một lượt chạy (run) của Dagster gặp thất bại, áp dụng theo dõi trên tất cả các Code Location.

b) Kích hoạt mỗi khi người dùng thực hiện chuyển đổi nhánh (branch deployment) trên GitHub.

c) Kích hoạt mỗi khi người dùng thực hiện chuyển đổi nhánh (branch deployment) trên GitHub.

d) Chỉ kích hoạt khi tỷ lệ dữ liệu bị từ chối vượt quá ngưỡng 50%.